

Agentic AI?

→ planner + executor + memory + verify.

LLM → large language models

Basic ML

→ regression, Classification



continuous

↳ classify

↳ Supervised & Unsupervised



input ↔ output

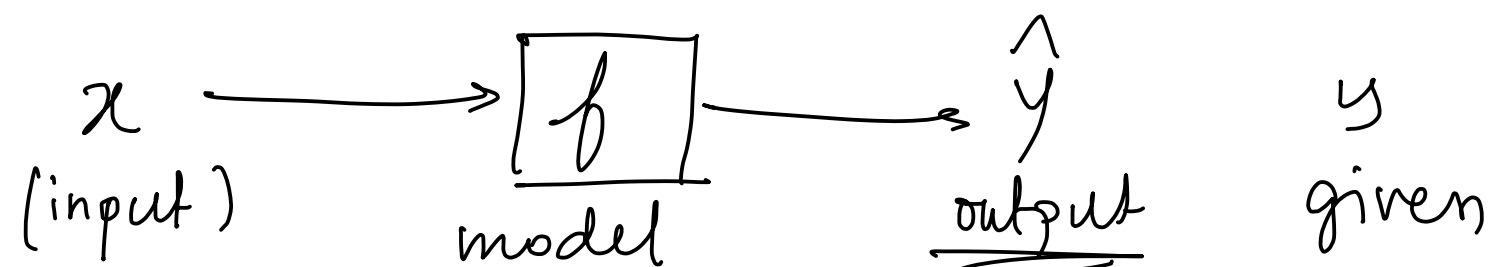


X output

Q] Price of a house:—

features :- size, location, floors

output :- price



Linear Regression

$$f_{w,b}(x) = \underline{w}x + \underline{b} = \begin{bmatrix} \hat{y} \\ y \end{bmatrix} \longleftrightarrow y$$

x $\rightarrow$ size house $\longleftrightarrow$ price $\rightarrow$ (m pairs)

$$\text{Error} \left\{ \begin{array}{l} \text{fun} \end{array} \right\} \underbrace{\sum (\hat{y}_i - y)^2}_{2m} \rightarrow 2m?$$

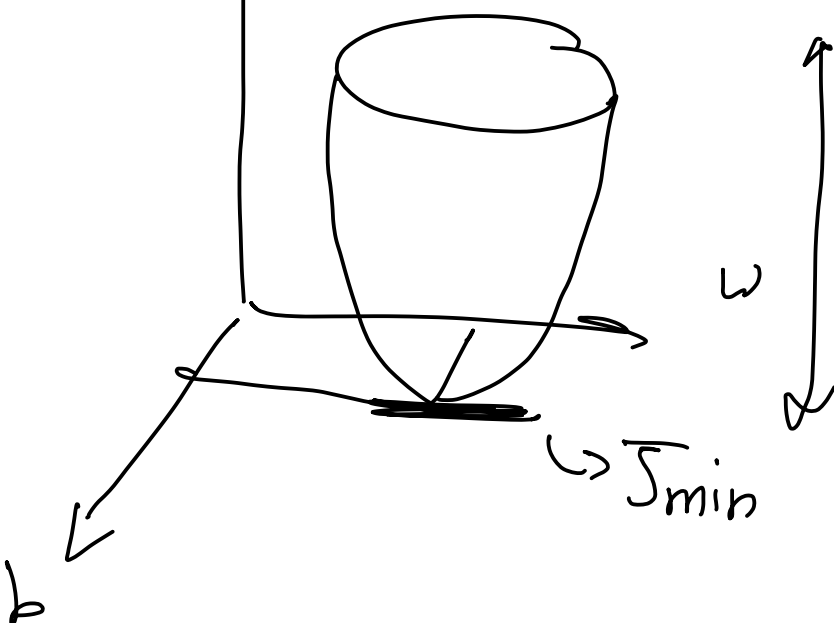
$$\hookrightarrow J(w, b)$$

$$J(w, b) = \frac{1}{2m} \sum ((w_i x_i + b_i) - y)^2$$

$J \rightarrow w, b$ quadratic

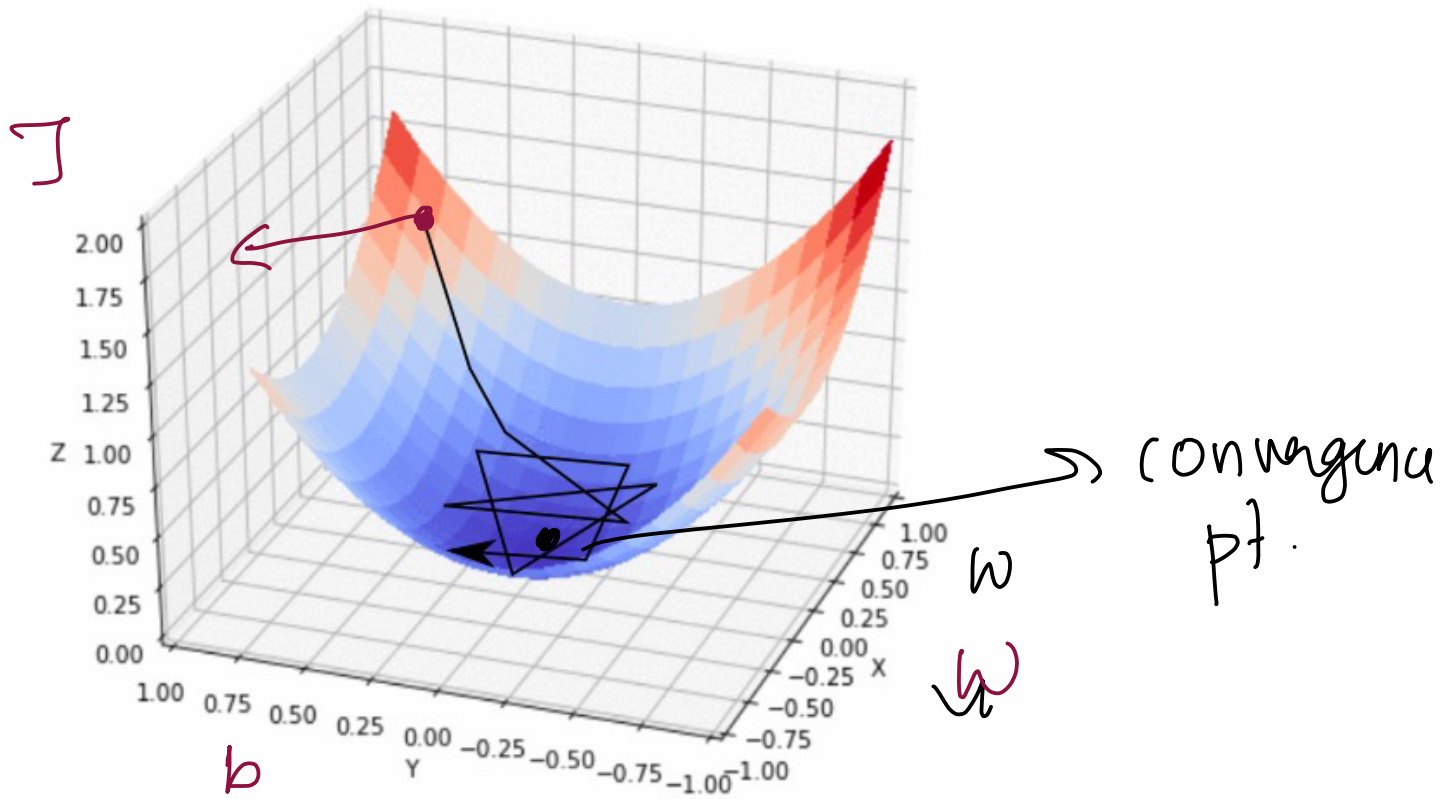
squared error

$\checkmark J$ error



$$y \approx \hat{y}$$

Gradient descent



adjusting value. $\rightarrow$ updating

$$w' = w - \alpha \cdot \frac{\partial J(w, b)}{\partial w}$$

$\alpha \rightarrow$ learning rate

$$b' = b - \alpha \cdot \frac{\partial J(w, b)}{\partial b}$$

$x \rightarrow$ 1 feature size $\rightarrow$ 1 out y price

Multiple features

$$f_{w,b}(x) = w_1 x_1 + w_2 x_2 - \dots + b$$

$$= (\vec{w} \cdot \vec{x} + b) \rightarrow \text{vectorization}$$

①

$$\underline{J(\vec{w}, b)}$$

$$\begin{matrix} w_j \\ \vdots \\ w_n \\ b \end{matrix} = w_j - \alpha \frac{\partial J(w_1, \dots, w_n, b)}{\partial w_j}$$

→ Price? size, location, floor

100 - 1000 place 1, 2

sq ↓ coded num

Feature Scaling

100 $\rightarrow$ 1000

1, 2, ..., 10

0 $\rightarrow$ 1, -1 $\rightarrow$ 1

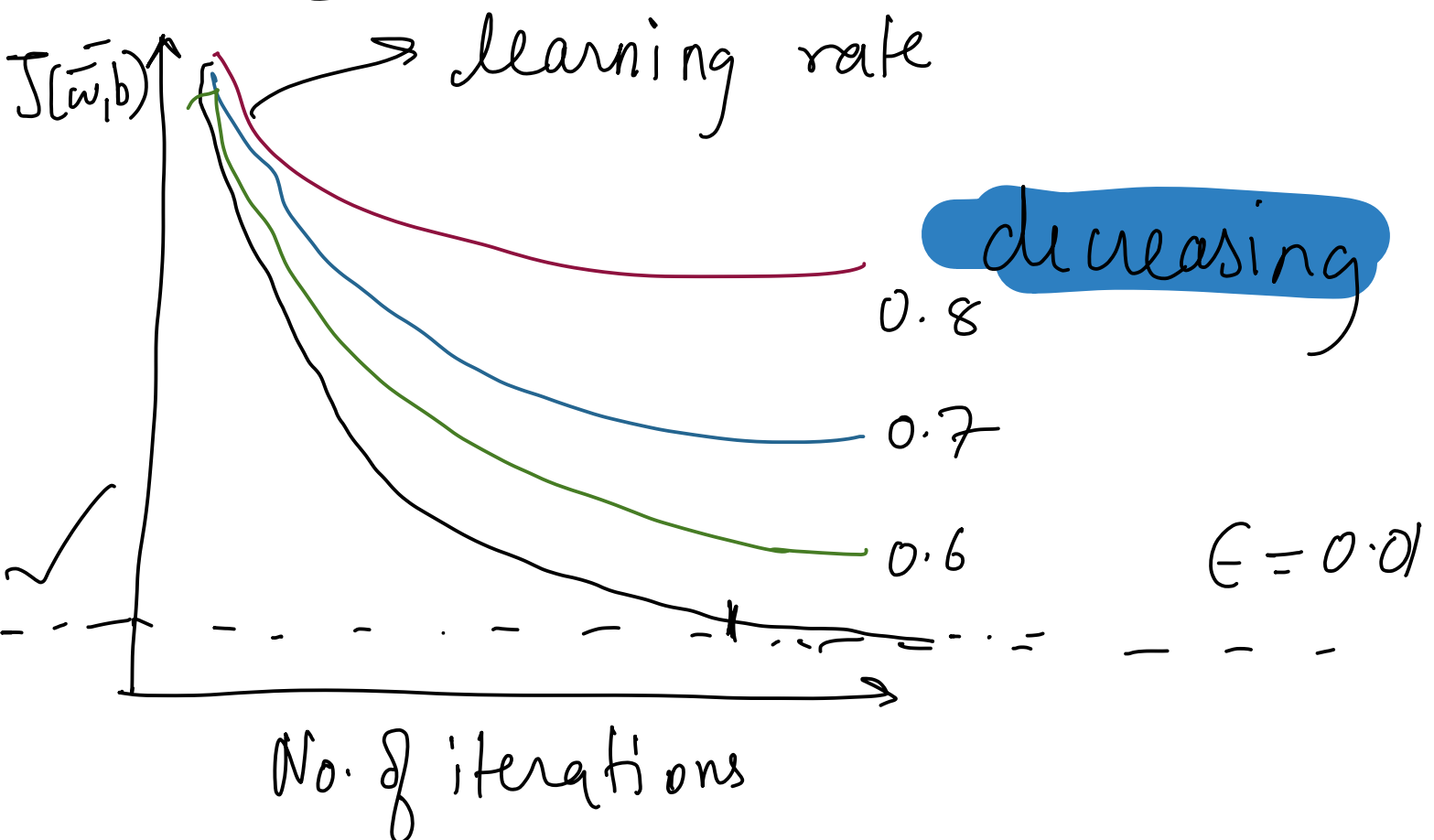
1) Normalization

(x) $\dots x^1, x^2, \dots, x^n$

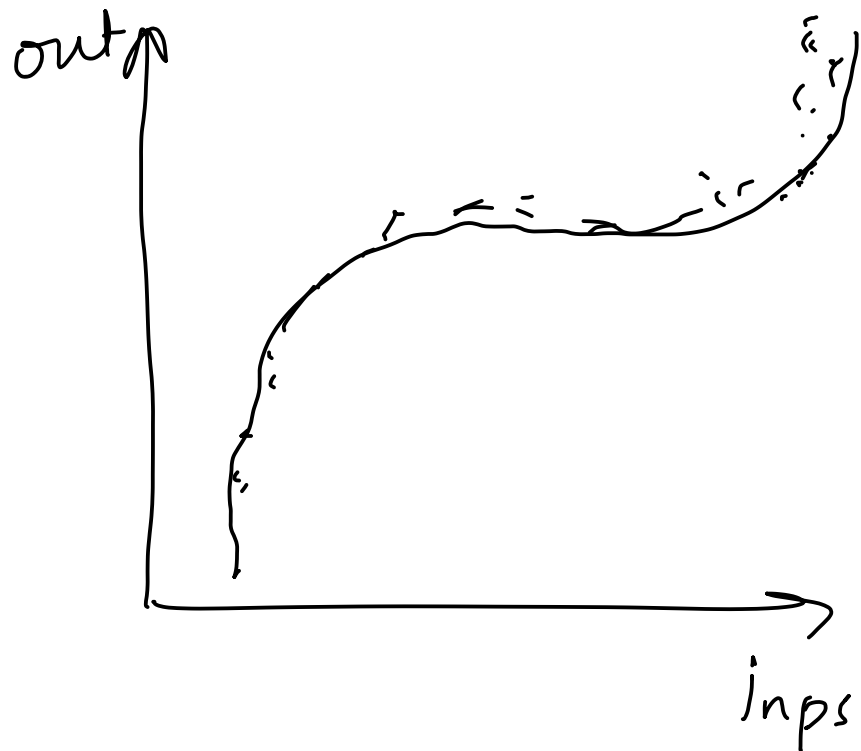
$$\bar{x}_1^i = \frac{x_1^i - \mu_1}{\sigma_1}$$

$$\bar{x}_2^i = \frac{x_2^i - \mu_2}{\sigma_2}$$

Learning curve



Polynomial regression



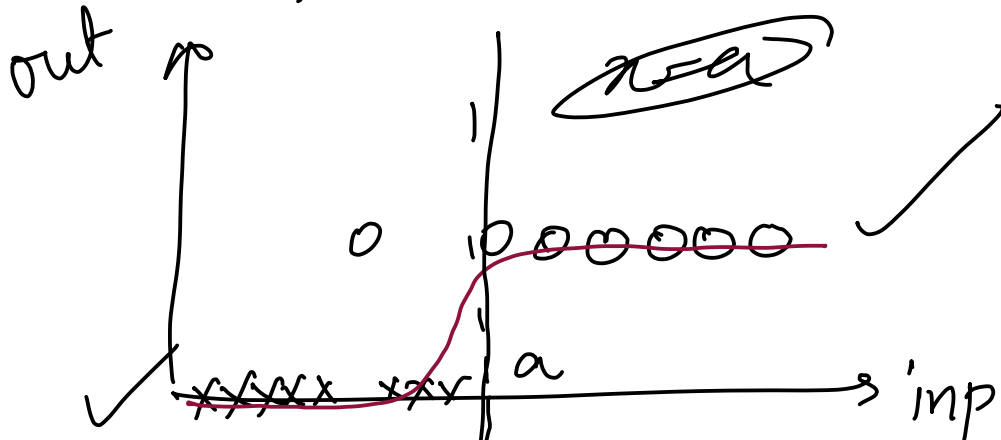
$$f_{\bar{w}, b}(x) = w_1 x + w_2 x^2 + w_3 x^3 + b$$

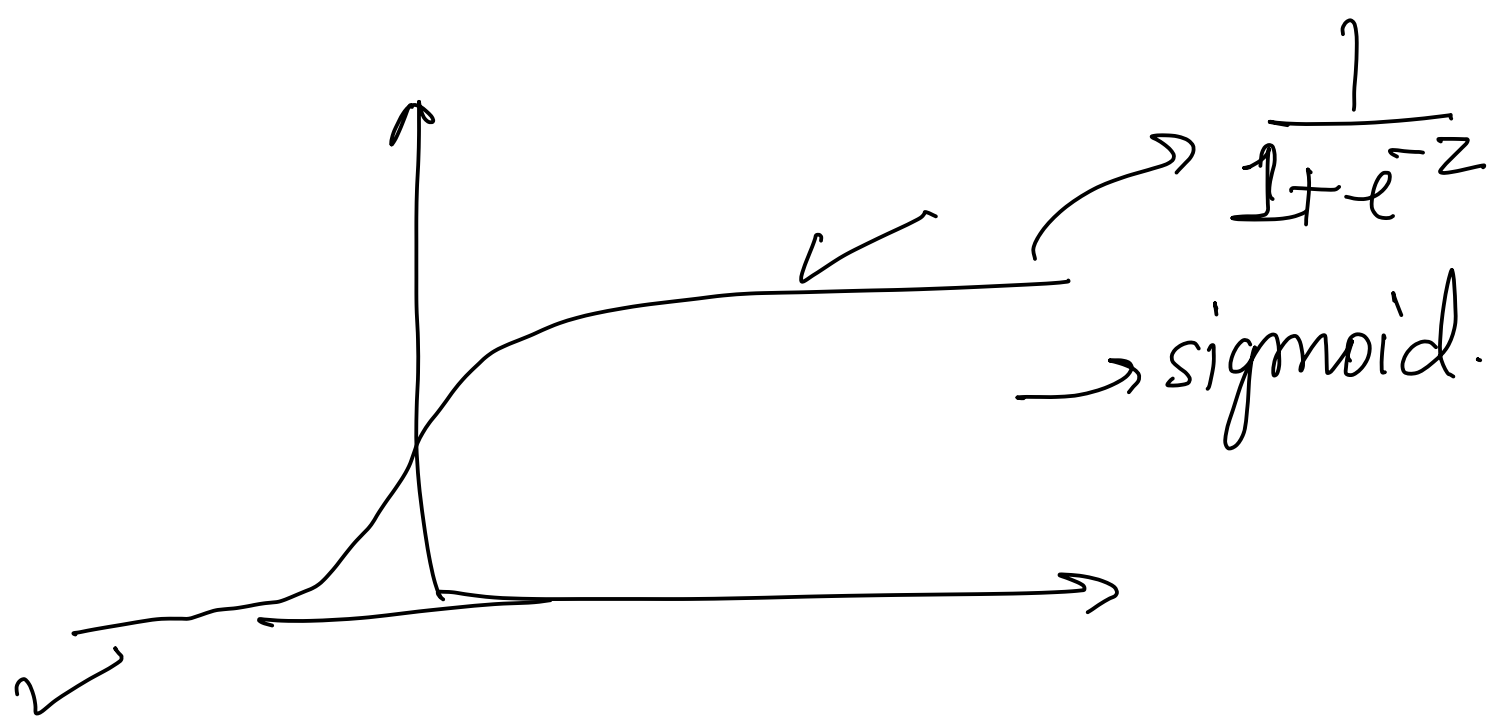
1 feature

logistic regression

→ classification task, T/F

→ output → 1, 0





$$f_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = z$$

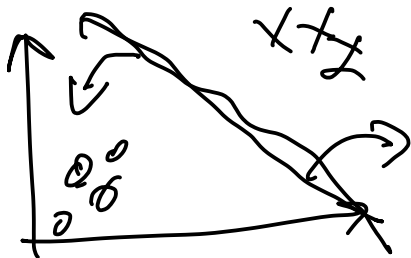
$$g(z) = \frac{1}{1+e^{-z}} \quad \checkmark$$

$$f_{\vec{w}, b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w} \cdot \vec{x} + b)}}$$

decision boundary

$$f_{\vec{w}, b}(\vec{x}) \geq a \longrightarrow y = 1$$

$$< a \longrightarrow y = 0$$



$$f_{\vec{w}, b}(\vec{x}) = g(z) = g(w_1 x_1 + w_2 x_2 + b)$$

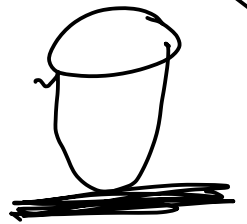
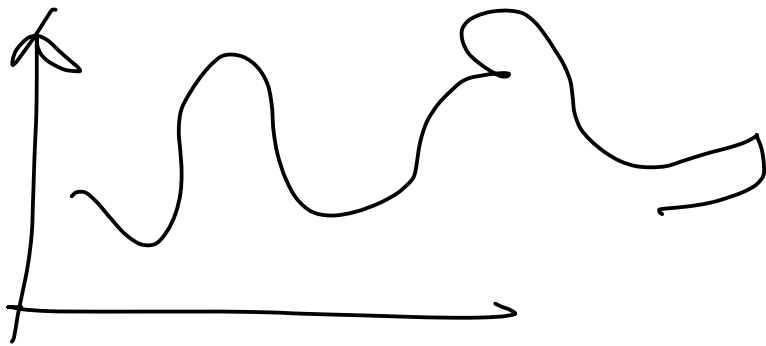
$\downarrow$ $\downarrow$ $\downarrow$

$$(z = x_1 + 2x_2 - 3 = 0) \rightarrow \text{decision boundary}$$

Cost fun

$$J = \frac{1}{1 - e^{-(\vec{w} \cdot \vec{x} + b)}} \quad y \rightarrow \text{quad } J$$

~~quad J~~



New cost fun

→ logistic loss fun

$$L(f_{\vec{w}, b}(x, y)) = \begin{cases} -\log(f_{\vec{w}, b}(x, 1)) & y=1 \\ -\log(1 - f_{\vec{w}, b}(x, 1)) & y=0 \end{cases}$$

$\rightarrow \underline{\underline{y=1}}$
 $\rightarrow \underline{\underline{y=0}}$

$$= -y^{(i)} \log(f_{w,b}(x^{(i)})) \\ - (1-y^{(i)}) \log(1-f_{w,b}(x^{(i)}))$$

$$J(\vec{w}, b) = -\frac{1}{m} \sum L(f_{w,b}(x^i), y^i)$$

$$\bullet w_j^o = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

linear
reg

$$f \rightarrow \vec{w} \cdot \vec{x} + b$$

logistic
reg

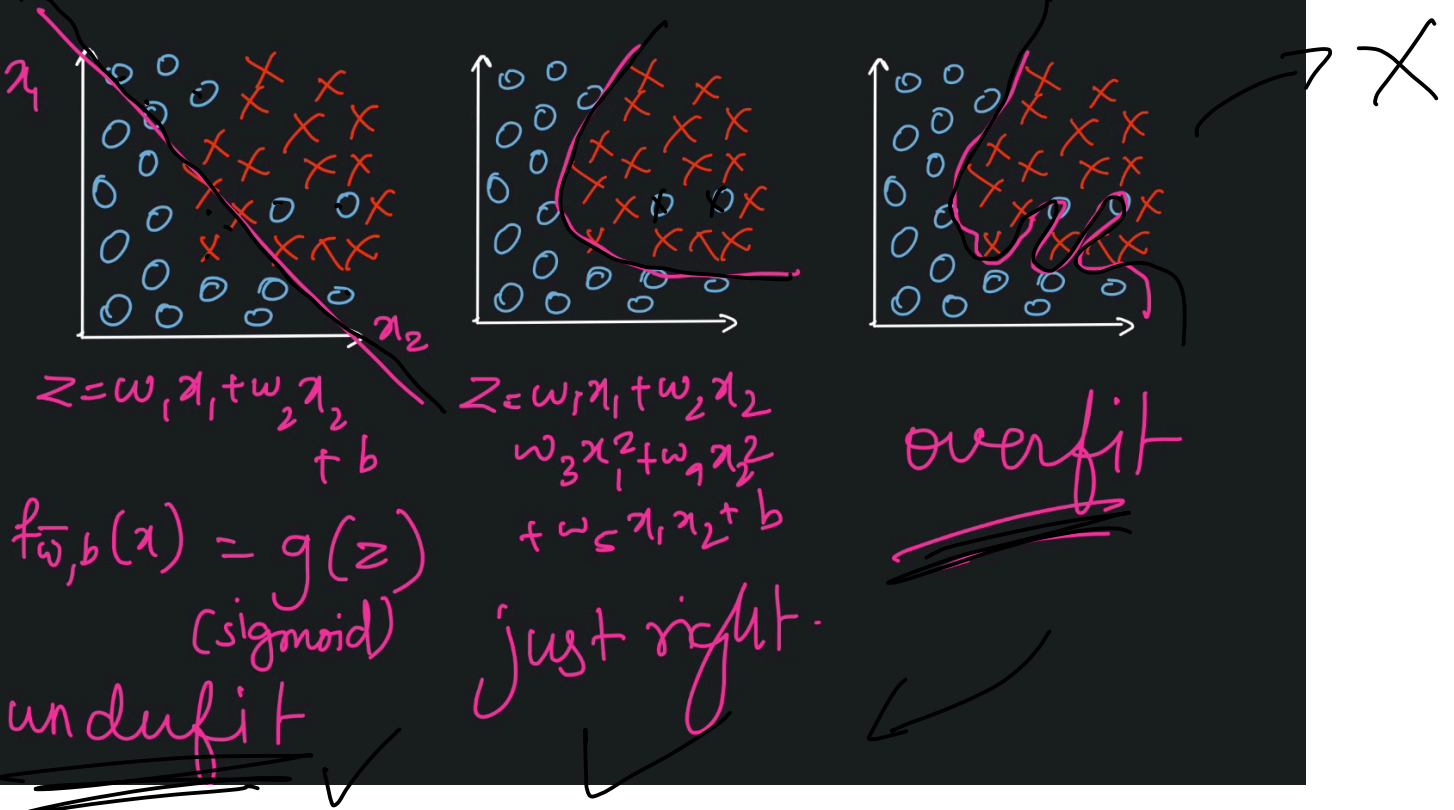
$$f = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

Overfitting & Underfitting

↳ high variance

↳ high bias

Eg) classification x_1, x_2 features



fix ??

Methods

- ① Collect more training data → underfit
- ② Reduce polynomial curves → over
- ③ Appropriate feature selection → both

④ Regularisation

→ removing feature X

→ reduce the weight given to it ✓

Cost fun

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^n (f_{w,b}(x_i^0) - y_i^0)^2 + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\text{penalty}}$$

exploding term.

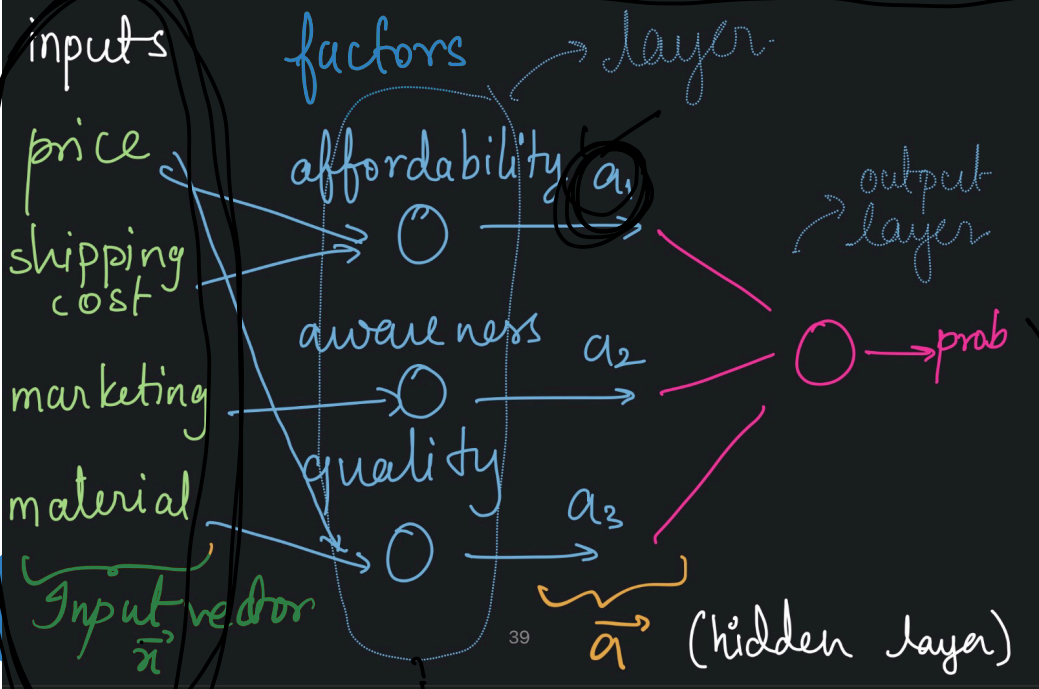
$w_i, x_i \rightarrow$ effect reduced.

L_1, L_2

$\rightarrow \sum w_j$

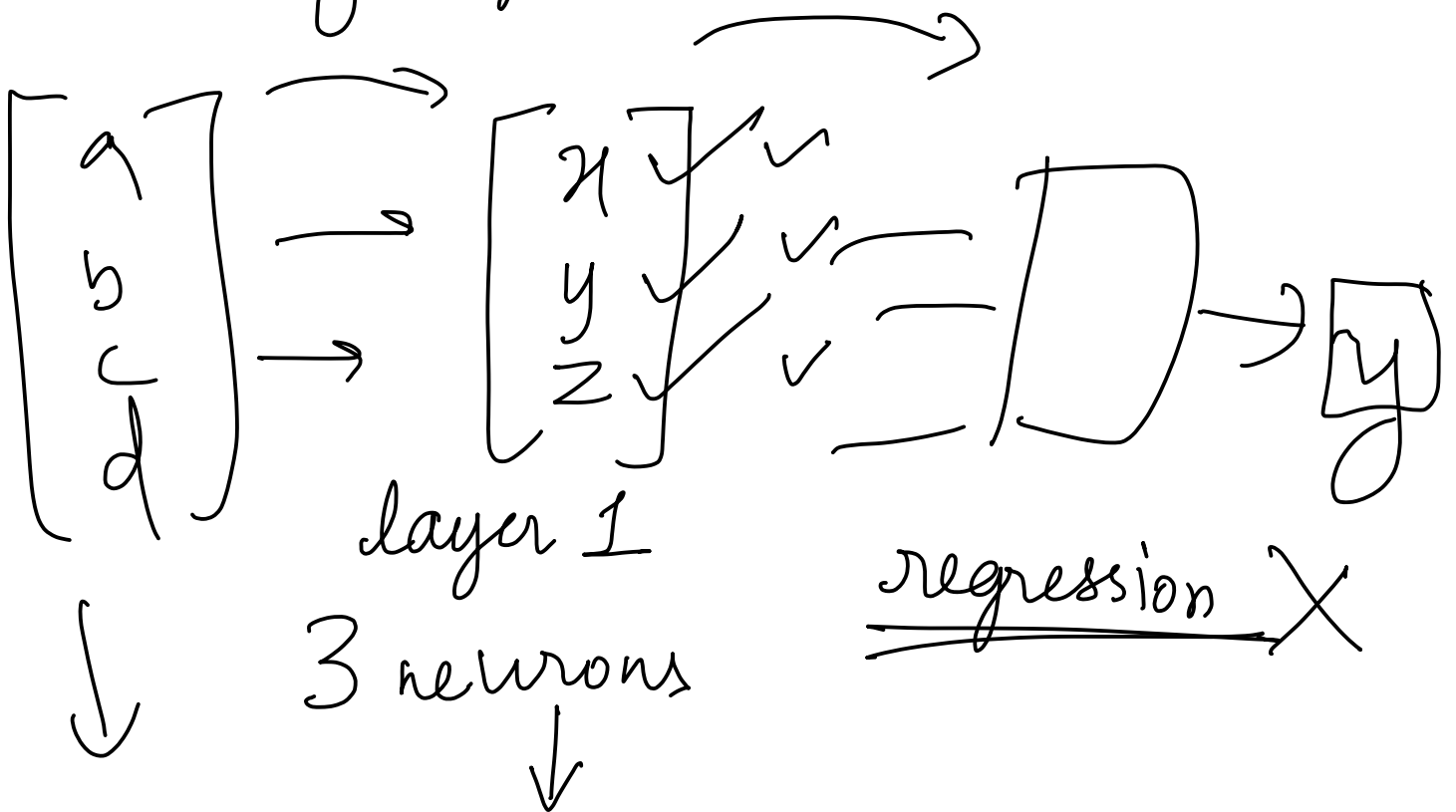
Neural Network

Ex) Probab of T-shirt being bestseller



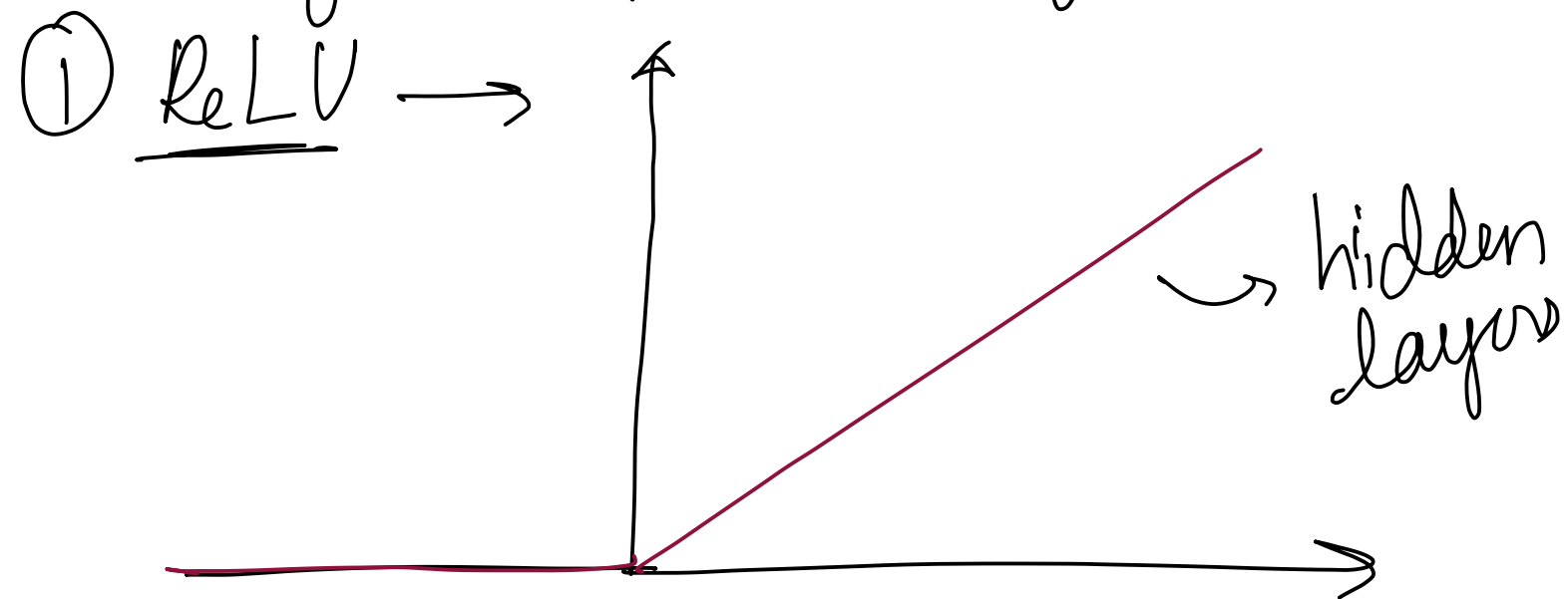
(layer $\rightarrow$ intermediate)

Bunch of regression



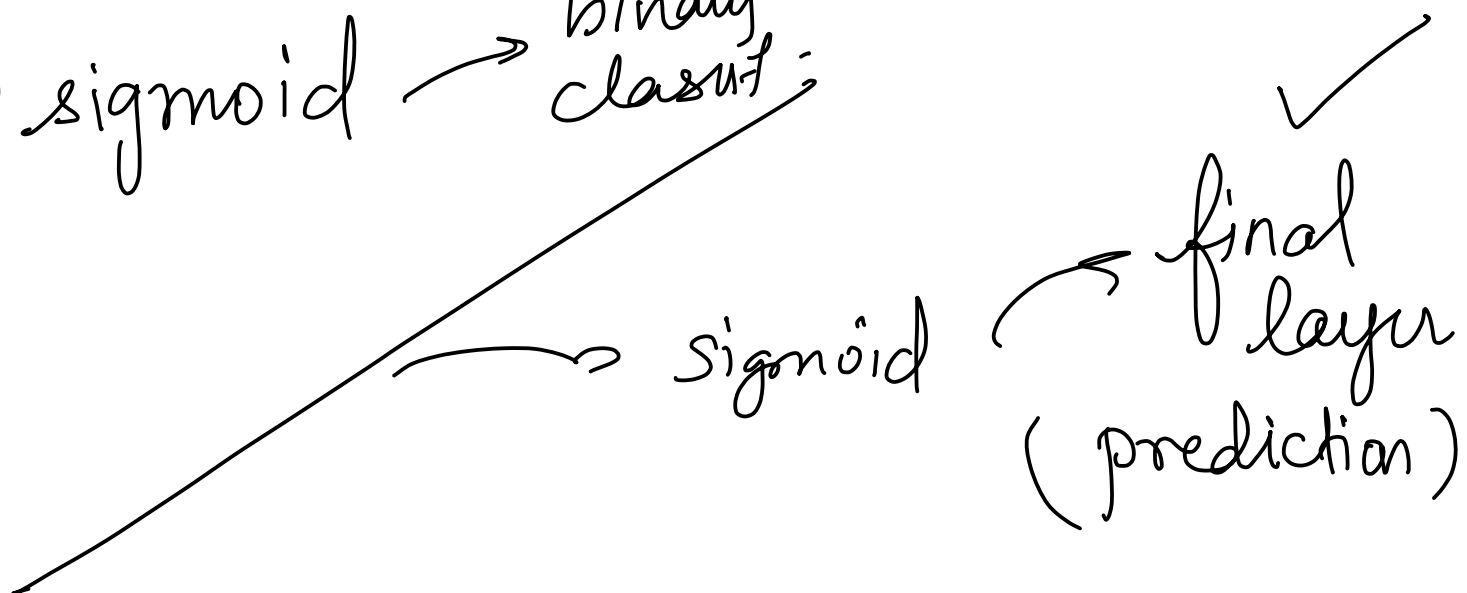
vectors vector
→ **Tensorflow** ✓

→ layer → activation function



$\text{max}(0, x)$

② sigmoid → binary class;



③ softmax

① More classifications

$$\left\{ \begin{array}{l} z_1 = \bar{w}_1 \cdot \bar{x}_1 + b_1 \rightarrow \text{class 1} \\ \vdots \\ z_4 = \bar{w}_4 \cdot \bar{x}_4 + b_4 \rightarrow \text{class 4} \end{array} \right.$$

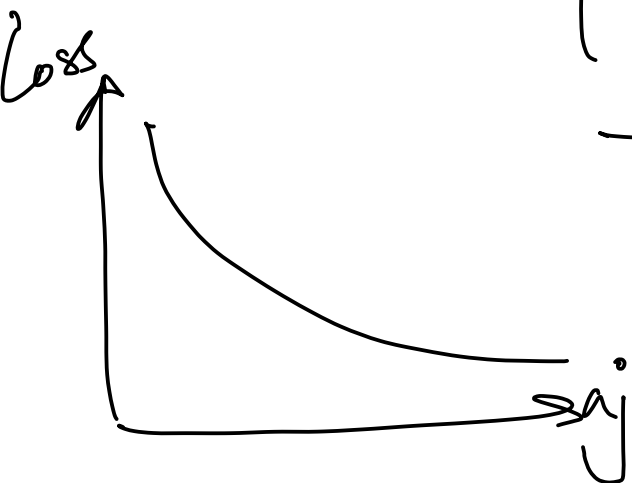
$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$$

$$a_j = \frac{e^{z_j}}{\sum e^{z_k}} \rightarrow \text{softmax layer}$$

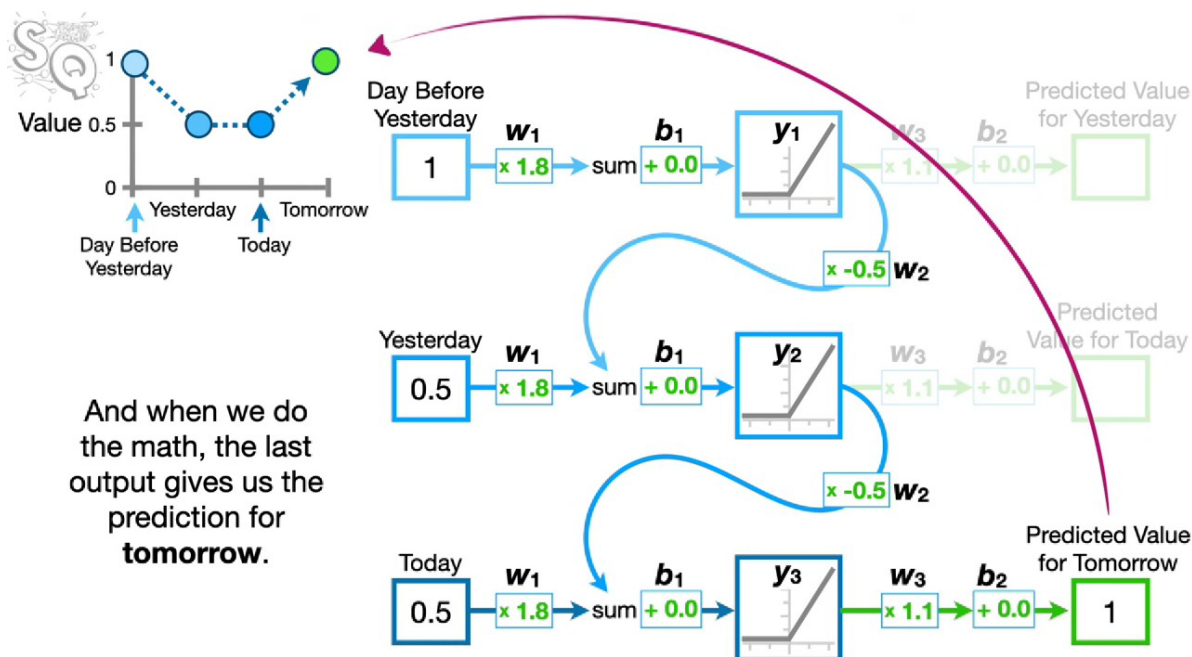
Cost $\rightarrow$

$$\text{loss}(a_1, \dots, a_n, y)$$

$$= \begin{cases} -\log a_1 & y=1 \\ -\log a_2 & y=2 \\ \vdots \\ -\log a_n & y=n \end{cases}$$



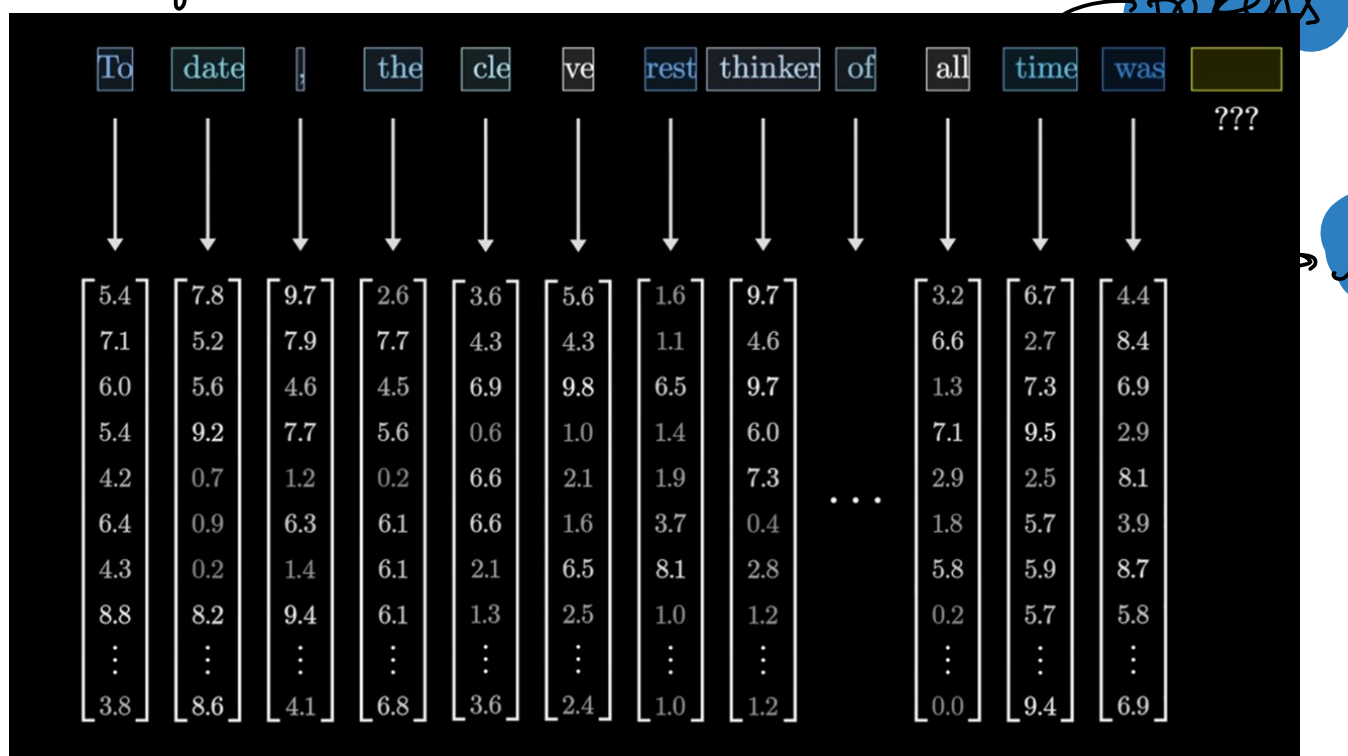
RNNs



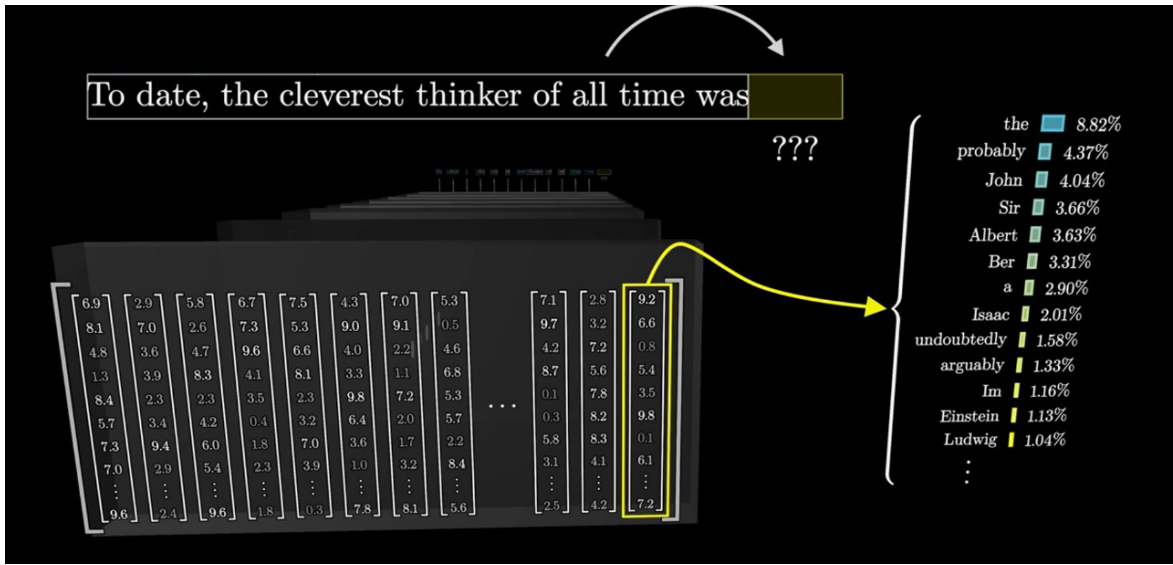
sequential data , customisable no. of inps.

LLMs (Attention mechanism)

→ Predict next word by taking all words in one go.

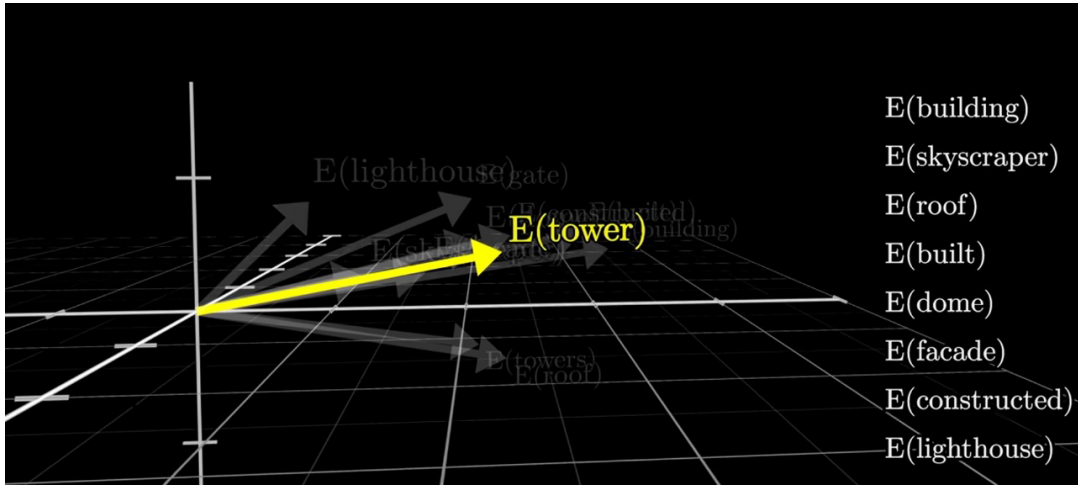
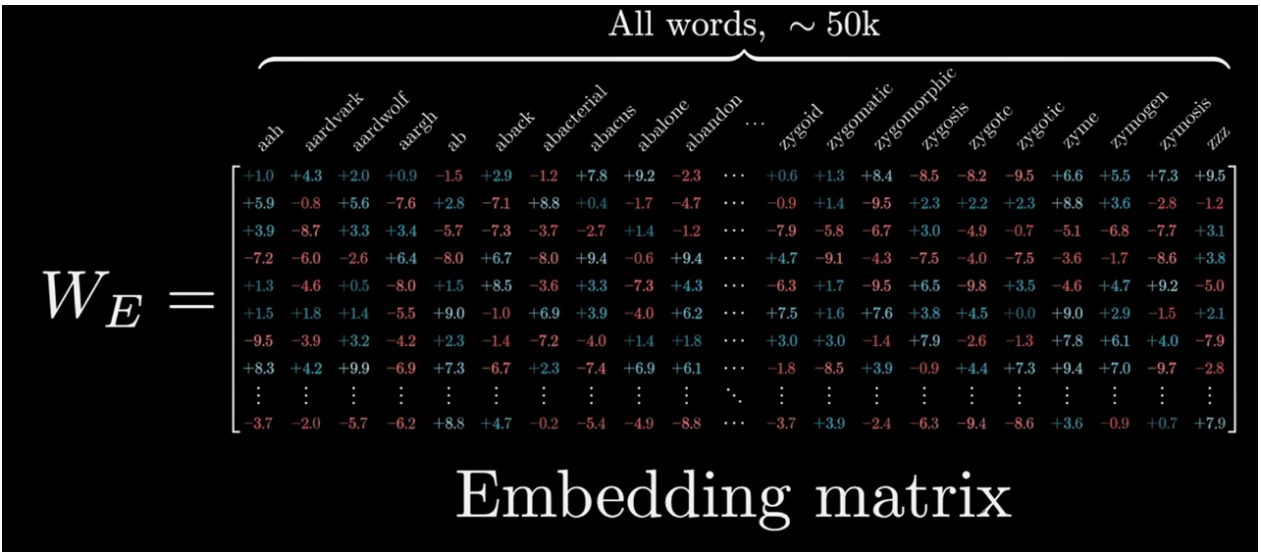


A machine learning model fashion model



semantic meaning

→ probability of words



semantic

$\vec{\text{vectors}} = \vec{\text{word}_1} - \vec{\text{word}_2}$

→ dot & cross product for Attention mechanism

→ context size

(distance)

